

为什么单机 Prefill/Decode 分离可能更慢

脱敏分享版 | 本文讨论基于 KV Cache 传输的 Prefill/Decode disaggregation 验证方法。服务、模型、端口、路径和基础设施信息均已匿名化。

结论先行

Prefill/Decode 分离的价值在于跨机部署、独立扩缩容和不同资源池的利用率，不保证单机低负载更快。一次 TP=1、`--enforce-eager`、单机 loopback、低负载的功能与性能验证中：

- 短请求 20 次对照，分离模式平均延迟增加 5.5%；
- 32 请求、并发 1/2/4/8 时，吞吐下降 3.3%–5.9%；
- 观察到 64 次 KV ready ACK，0 次 missing transfer id；
- 在约 3.6–14.6 MiB KV 范围内，Decode 侧“拉取 KV + 固定长度生成”约 0.231 s，随 prompt 变化不明显。

这些证据说明 transfer-id 关联协议连续、P→D transfer path 被触发，且 D 段行为与复用 P 侧 KV 的预期一致；在当前观测粒度下，该量级传输未成为 D 段的可见主导项。由于没有 KV checksum 和实际传输字节统计，它们不能证明 payload 内容与字节数正确，也不证明单独 RDMA latency 为 0.231 s，更不证明 P/D 在 CUDA Graph、多 TP、跨机或大规模下必然提升吞吐。单机对照中 baseline 占一组 GPU，而 P/D 占两组 GPU，也不是等资源效率比较。

1. P/D 分离解决什么问题

LLM 生成可以拆成两类资源特征不同的阶段：

Prefill：处理完整 prompt，矩阵规模大，算力密集
Decode：逐 token 生成，频繁读取 KV Cache，延迟与带宽敏感

P/D disaggregation 将两个阶段放到独立 worker：

```
Client
-> Prefill worker: prompt -> KV Cache
-> KV transfer engine
-> Decode worker: imported KV -> new tokens
```

这样可以分别扩容 P 和 D，并让硬件配置适配各自负载。但拆分也引入新的固定成本：两段请求、会话建立、元数据握手、KV 传输和额外排队。

2. 服务启动不等于真正分离

一个最常见的假阳性是：P、D 两个服务都正常返回，但 D 并未复用 P 产生的 KV，而是重新执行了 prefill。

2.1 Transfer ID 是状态协议

P 和 D 必须通过同一 transfer id 关联：

```
request id / transfer id
|
├─ P: compute KV, publish location
└─ D: query location, fetch KV, skip local prefill
```

如果代理或网关没有把 transfer id 注入第二段请求，D 无法找到远端 KV，只能失败或回退到本地 prefill。此时“请求成功”不能证明 disaggregation 生效。

2.2 三类真实性证据

证据	说明
Ready ACK	P 已发布、D 已确认可拉取的 KV block
Missing-ID counter	检查关联协议是否丢失
Prompt sweep 的阶段耗时	判断 D 是否随 prompt 重新做 prefill

受测运行中有 64 个 ready ACK，missing transfer id 为 0，说明关联协议连续有效且 transfer path 被触发；这两项计数本身不校验 KV payload 内容或实际传输字节数。

3. 延迟要拆成 P 段和 D 段

总延迟可近似写成：

$$T_{\text{total}} = T_{\text{request_P}} + T_{\text{prefill}} + T_{\text{publish}} \\ + T_{\text{request_D}} + T_{\text{fetch_KV}} + T_{\text{decode}}$$

单机短请求对照中，baseline 平均延迟归一化为 1.000，P/D 为 1.055。增加的 5.5% 包含跨进程请求、状态握手和 KV 传输等固定成本，不能只归因于 RDMA。

此外，D 段测到的是 `T_fetch_KV + T_decode`，并非纯传输延迟。若生成长度固定，D 段约 0.231 s 且随 prompt 基本不变，只能说明在当前 TP=1、`--enforce-eager`、单机 loopback 与观测粒度下，传输未成为可见主导项。

4. 用 Prompt Sweep 判断 D 是否重算 Prefill

如果 D 重新执行 prefill，D 段应随 prompt 长度明显增加。真实 KV 复用时：

```
prompt grows
P prefill: increases
D fetch + fixed decode: approximately stable
```

受测范围内，KV 规模从约 3.6 MiB 增加到 14.6 MiB，P 段随 prompt 增长，而 D 段保持约 0.231 s。这与“P 负责 prefill、D 只拉 KV 并 decode”的路径一致。

注意：这仍是间接分段证据。更严格的验证还可以加入 D 侧 prefill kernel counter、KV block checksum 和传输字节统计。

5. 并发对照为什么仍是负收益

在同一 TP=1、`--enforce-eager`、单机 loopback 条件下，32 个请求、并发 1/2/4/8 时，P/D 吞吐相对 baseline 下降 3.3%–5.9%。原因包括：

- 单机 loopback 下没有跨资源池优势；
- 每个请求多一次服务边界和状态关联；
- P、D 各保留一套模型与 KV pool；
- 低并发无法摊薄固定成本；
- Decode 仍是主要耗时，P 阶段没有形成可独立扩缩容的压力。

低负载单机：额外协议成本 > 独立扩缩容收益

跨机高负载：独立扩缩容收益 可能 > 协议与传输成本

“可能”非常重要，后者仍需专门实验，不能由单机结果推断。

6. 资源口径：不是等卡比较

Baseline 使用一组 GPU，P/D 使用两组 GPU。即使请求吞吐相同，P/D 的资源效率也可能更低。因此至少应同时报告：

- req/s 与 output tok/s；
- 每 GPU 吞吐；
- P、D 各自利用率；
- KV pool 与模型副本显存；
- 排队时间；
- 网络吞吐与传输字节；
- 端到端功耗或成本。

本次验证只适合回答“关联协议是否连续、transfer path 是否被触发、D 段行为是否符合 KV reuse 预期”和“该低负载配置下的固定成本是多少”，不适合证明 payload 正确性或比较规模化资源效率。

7. 一套更完整的 P/D 验证矩阵

7.1 功能层

- transfer id 连续一致；
- ready ACK 与传输 block 数匹配；
- 0 missing id；
- D 不重算 prefill；
- 断连、超时和 session 恢复；
- 多请求并发时 KV 不串线。

7.2 正确性层

- baseline/P-D 的生成 token；
- KV block checksum；
- 不同 prompt、长度、batch；
- prefix cache hit/miss；
- sampling 与确定性配置分开。

7.3 性能层

维度	建议扫描
部署	单机、跨机、同/跨交换域
负载	短 prompt、长 prompt、短/长 decode
并发	低、中、高与突发
比例	不同 P:D worker 配比
指标	TTFT、TPOT、E2E、req/s、tok/s、每 GPU 吞吐

7.4 扩缩容层

真正的价值验证需要固定总资源或成本，对比：

- 统一 worker 池；
- 独立 P/D worker 池；
- 动态改变 P:D 比例；
- 请求分布变化下的队列长度和 SLO。

8. 正确表述负结果

可以说：

- transfer-id 协议连续，P→D transfer path 已被触发，D 段行为与 KV reuse 预期一致；

- 当前 TP=1、`--enforce-eager`、单机 loopback 和 KV 规模下，传输未成为 D 段的可见主导项；
- 该低负载配置下存在约 4%–6% 固定性能成本。

不能说：

- RDMA 单独耗时就是 0.231 s；
- 没有 checksum 与传输字节统计时，KV payload 内容与字节数已经验证正确；
- P/D 已提升吞吐；
- 单机两组 GPU 与 baseline 一组 GPU 是等资源对照；
- 结果可外推跨机、高并发或独立扩缩容。

9. 可复用检查表

- ☐ P、D 是否共享同一 transfer id？
- ☐ 是否有 ready ACK、missing-id 和实际字节证据？
- ☐ D 是否可能静默重算 prefill？
- ☐ 是否把 D 段与纯 KV transfer latency 区分？
- ☐ prompt sweep 是否符合 P 增长、D 稳定的预期？
- ☐ baseline 与 P/D 是否使用相同总资源？
- ☐ 是否报告每 GPU 吞吐和模型副本成本？
- ☐ 是否明确 TP、eager/Graph、loopback/跨机与负载边界？
- ☐ 是否把单机功能 benchmark 与跨机扩缩容价值分开？

10. 总结

P/D 分离首先是一个跨进程状态协议，其次才是性能优化。验证时必须区分“关联协议连续、transfer path 被触发”和“payload 已被 checksum/字节统计证明正确”，再讨论吞吐。本次 TP=1、`--enforce-eager`、单机 loopback、低负载实验只支撑前一层证据；其 3%–6% 负收益明确了当前配置的固定成本，不能外推 CUDA Graph、多 TP、跨机或高并发。只有在跨机、高负载或 P/D 资源需求显著失衡的独立实验中，才能判断扩缩容收益能否覆盖这笔成本。